

Kalshi Crypto Trading Bot — Technical Whitepaper

Marina Kagan

February 2026

Contents

Part 1: Executive Summary	2
What It Does	2
Market Opportunity	2
Strategy in Plain English	2
Key Differentiators	3
Part 2: System Architecture	3
Data Pipeline	3
Component Overview	3
Data Sources	4
Part 3: Technical Deep-Dive	5
3.1 Volatility Engine	5
Realized Kernel (Barndorff-Nielsen 2008)	5
HAR-RV Blending	5
Jump Detection	5
DVOL Integration	6
Cross-Asset Beta	6
3.2 Probability Model	6
Step 1: Z-Score Computation	6
Step 2: Student-t CDF (df=4)	6
Step 3: Logistic Calibration	7
Step 4: Dynamic Probability Cap	7
Step 5: Market-Price Blending	7
Sanity Checks	7
3.3 Edge Detection	7
Fee Formula	7
Fee-Adjusted Edge	8
3.4 Order Flow Analysis	8
Cross-Exchange Consensus	8
Funding Rate Signals	8
Total Adjustment Cap	8
3.5 Execution Strategy	8
Execution Modes	8
Maker-to-Taker Escalation	9

UUID Persistence	9
3.6 Position Sizing	9
Quarter-Kelly Formula	9
Drawdown Scaling	9
Part 4: Risk Management	10
Position Sizing Controls	10
Market Selection Controls	10
Model Sanity Controls	10
Execution Controls	10
Part 5: Performance & Observations	10
Overview	11
Filter Pipeline Breakdown	11
Hypothetical Performance	11
Win Rate by Price Bucket	11
Part 6: Infrastructure	11
Deployment	11
Data Persistence	12
SQLite (state.db)	12
JSONL Journals	12
Firebase Real-Time Dashboard	12
Observation Mode	12

Part 1: Executive Summary

What It Does

This system is an automated trading bot for **Kalshi**, a CFTC-regulated prediction market exchange. It trades **15-minute cryptocurrency price threshold contracts** — binary options that pay \$1 if a crypto asset (BTC, ETH, SOL, or XRP) stays above a given price at the end of a 15-minute window, and \$0 otherwise.

The bot monitors real-time prices across multiple exchanges, estimates the probability of each outcome using microstructure-aware volatility models, and places trades when its model identifies a statistical edge over the market price.

Market Opportunity

Kalshi lists 15-minute crypto contracts around the clock. Each window produces fresh contracts for four assets at multiple strike prices, creating hundreds of tradeable markets per day. Because these are short-duration, binary-outcome instruments, mispricing tends to be small but frequent — an ideal environment for systematic, model-driven trading.

Strategy in Plain English

1. **Observe** — Continuously stream spot prices from Coinbase, Binance, Kraken, and Bybit. Fetch implied volatility from Deribit and funding rates from CoinGlass.

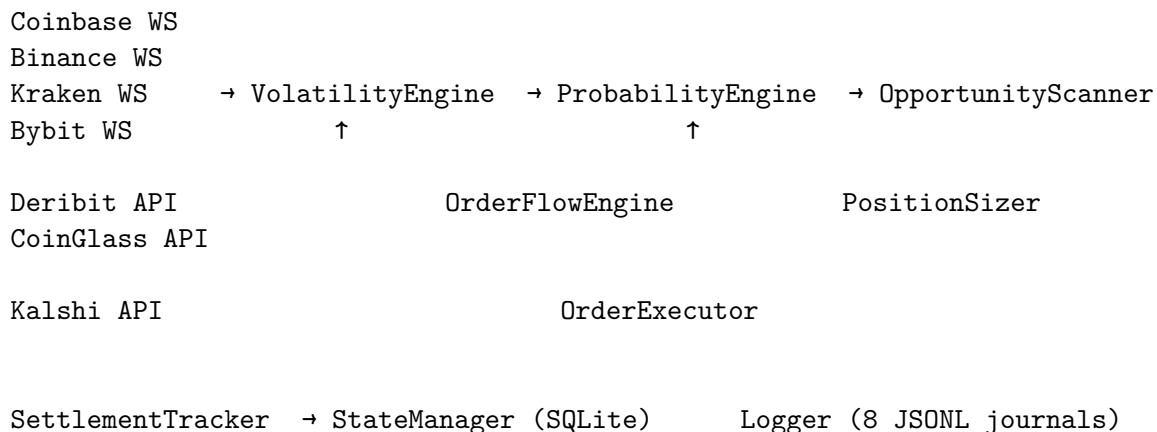
2. **Estimate** — For every active market, compute the probability that the asset stays above its threshold using a volatility-weighted, fat-tailed statistical model.
3. **Filter** — Reject markets that are too uncertain, too expensive, or offer insufficient edge after fees.
4. **Size** — Use a conservative Kelly criterion (quarter-Kelly) to determine position size, with automatic scaling during drawdowns.
5. **Execute** — Place maker (limit) orders first to minimize fees, escalating to taker orders if time runs short.
6. **Settle** — Track outcomes via the Kalshi settlements API and log performance for continuous evaluation.

Key Differentiators

- **Multi-exchange intelligence:** Aggregates spot prices from 4 exchanges plus derivatives signals from Deribit and CoinGlass, detecting cross-exchange lead-lag patterns before they appear in Kalshi prices.
- **Microstructure-aware volatility:** Uses Realized Kernel estimation (Barndorff-Nielsen 2008) rather than naive sample variance, correctly handling market microstructure noise.
- **Adaptive execution:** Maker-first strategy with time-aware taker escalation minimizes fees while ensuring fills before window expiry.
- **Comprehensive risk controls:** Quarter-Kelly sizing, drawdown scaling, single-asset-per-window rule, z-score sanity checks, and model-market discrepancy detection.

Part 2: System Architecture

Data Pipeline



Component Overview

The system comprises 15 classes, each with a single responsibility:

Component	Role
KalshiClient	API communication with RSA-PSS authentication and per-second rate limiting (30 reads/sec, 30 writes/sec on Advanced tier)
Logger	Structured JSONL logging across 8 journals with fill deduplication
StateManager	SQLite-backed persistent state (WAL mode for crash resilience); tracks positions, orders, fills, and settlements
CoinbaseFeed	Real-time WebSocket feed for BTC, ETH, SOL, XRP with 300-point price buffer (5 minutes at 1-second intervals)
DeribitDVOLFetcher	Daemon thread fetching implied volatility (DVOL) index for BTC and ETH every 60 seconds
CrossExchangeFeed	Multi-exchange WebSocket feeds from Binance, Kraken, and Bybit for cross-exchange lead-lag detection
CoinGlassFetcher	Funding rate data from CoinGlass API (10-minute intervals, 15-minute TTL) for leverage regime detection
OrderFlowEngine	Aggregates cross-exchange consensus and derivatives signals into probability adjustments (capped at ± 3 percentage points)
VolatilityEngine	Realized Kernel volatility with HAR-RV blending across 1/5/15-minute windows, plus jump detection and DVOL integration
ProbabilityEngine	Win probability via Student-t CDF (df=4) with logistic calibration, dynamic caps, and market-price blending
PositionSizer	Quarter-Kelly position sizing with drawdown-based scaling
OpportunityScanner	10-stage filter pipeline evaluating all markets across active 15-minute windows
OrderExecutor	Maker-first limit orders with adaptive taker escalation based on time-to-expiry
SettlementTracker	Incremental settlement polling (30-second intervals) using the Kalshi settlements API
MainLoop	Continuous 1-second observation loop coordinating all components

Data Sources

Source	Data	Transport	Frequency
Coinbase	Spot prices (BTC, ETH, SOL, XRP)	WebSocket	Real-time (1s snapshots)
Binance	Spot prices (cross-exchange)	WebSocket	Real-time
Kraken	Spot prices (cross-exchange)	WebSocket	Real-time
Bybit	Spot prices (cross-exchange)	WebSocket	Real-time
Deribit	Implied volatility (DVOL) for BTC/ETH	REST API	60 seconds
CoinGlass	Funding rates (BTC, ETH, SOL, XRP)	REST API	10 minutes
Kalshi	Markets, orderbooks, balance, fills, settlements	REST API	On-demand

Part 3: Technical Deep-Dive

3.1 Volatility Engine

The volatility engine produces a per-asset, per-5-second realized volatility estimate that feeds the probability model. It combines three techniques:

Realized Kernel (Barndorff-Nielsen 2008)

Standard sample variance of high-frequency returns is biased by market microstructure noise (bid-ask bounce, discrete tick sizes). The Realized Kernel estimator corrects this using a kernel-weighted autocovariance function, producing noise-robust volatility estimates from 5-second log returns.

HAR-RV Blending

Volatility exhibits heterogeneous persistence — recent moves matter more than distant ones, but ignoring longer history causes whipsawing. The system computes realized volatility at three horizons and blends them:

$$\sigma_{blended} = 0.50 \times \sigma_{1min} + 0.30 \times \sigma_{5min} + 0.20 \times \sigma_{15min}$$

where each σ_h is the square root of the sum of squared 5-second log returns over that horizon (12, 60, and 180 returns respectively).

Jump Detection

Jumps — sudden, large price moves — invalidate smooth volatility assumptions. The system detects them using a Bipower Variation comparison:

- A return exceeding 3σ of the current realized volatility triggers a **jump event**
- During a jump event, the volatility estimate is multiplied by **2.0** for **60 seconds**
- The elevated regime decays after 60 seconds, reverting to normal estimation

DVOL Integration

When Deribit implied volatility (DVOL) exceeds realized volatility by more than 50%, the system blends in the implied estimate:

$$\sigma_{final} = 0.75 \times \sigma_{RV} + 0.25 \times \sigma_{IV}$$

This respects the market's forward-looking information during regime changes while anchoring to observed data.

Cross-Asset Beta

For assets without direct DVOL data (SOL, XRP), the system estimates a cross-asset beta against BTC using a 60-return lookback window, allowing derivative signals to propagate across correlated assets.

3.2 Probability Model

Given the blended volatility, the probability engine estimates the likelihood that an asset stays above its threshold for the remaining window duration.

Step 1: Z-Score Computation

$$z = \frac{\text{threshold} - \text{spot}}{\text{spot} \times \sigma_{blended} \times \sqrt{T/5}}$$

where T is seconds remaining and $\sigma_{blended}$ is per-5-second scale. The denominator represents the expected magnitude of price movement over the remaining window.

Step 2: Student-t CDF (df=4)

Rather than assuming Gaussian returns, the model uses a Student-t distribution with 4 degrees of freedom:

$$p_{raw} = 1 - F_t(z; \nu = 4)$$

The fat tails of the Student-t distribution (kurtosis = 9 vs. 3 for Gaussian) better capture the empirical distribution of crypto returns, where large moves occur more frequently than a normal distribution would predict.

Step 3: Logistic Calibration

Raw probabilities are compressed toward 50% using a logistic function with slope $\beta = 0.85$:

$$p_{cal} = \text{logistic}(\beta \times \text{logit}(p_{raw}))$$

This calibration step accounts for model uncertainty — when $\beta < 1$, extreme probabilities are pulled toward the center, reflecting the reality that a model with finite data should not be maximally confident.

Step 4: Dynamic Probability Cap

A time-dependent cap prevents overconfidence:

Time to Expiry	Cap
0–30 seconds	93%
30–60 seconds	92%
60–300 seconds	91%
> 300 seconds	90%

Longer time horizons carry more uncertainty, justifying lower caps.

Step 5: Market-Price Blending

For market prices below 96¢, the calibrated probability is blended with the market-implied probability:

$$p_{final} = 0.50 \times p_{cal} + 0.50 \times p_{market}$$

At 96¢ and above, blending is skipped to preserve edge in high-confidence endgame scenarios.

Sanity Checks

- **Z-score limit:** If $|z| > 8$, the market is refused (model inputs are unreliable at extremes)
- **Model-market discrepancy:** If $p_{cal} > 90\%$ but market price $< 75\text{¢}$, the market is refused (suggests the model may be missing information the market has)

3.3 Edge Detection

A trade requires positive expected value after accounting for fees.

Fee Formula

Kalshi charges taker fees using a variance-based formula:

$$\text{fee} = \lceil 0.07 \times C \times P \times (1 - P) \rceil \text{ cents}$$

where C is the number of contracts and P is the trade price as a decimal. The ceiling is applied to the total, not per contract. Maker fees use 0.0175 instead of 0.07.

Fee-Adjusted Edge

$$\text{edge} = p_{\text{final}} - \frac{\text{best_ask}}{100} - \frac{\text{taker_fee}}{100}$$

A trade must satisfy:

$$\text{edge} \geq \text{MIN_EDGE_PCT} = 0.25\%$$

3.4 Order Flow Analysis

The OrderFlowEngine combines two signal sources to adjust the base probability:

Cross-Exchange Consensus

Real-time prices from Binance, Kraken, and Bybit are compared against Coinbase:

Signal	Condition	Adjustment
Strong consensus (favorable)	3+ exchanges lead by 0.3%	+2 pp
Single-exchange lead (favorable)	1 exchange leads by 0.2%	+1 pp
Strong consensus (opposing)	3+ exchanges oppose by 0.3%	-2 pp

Funding Rate Signals

Perpetual futures funding rates from CoinGlass indicate leverage buildup:

Signal	Condition	Adjustment
Extreme funding	Rate > 0.05% per 8h	-1.5 pp
Elevated funding	Rate > 0.03% per 8h	-0.5 pp

Total Adjustment Cap

All adjustments are summed and capped at **± 3 percentage points**, preventing any single signal source from dominating the probability estimate.

3.5 Execution Strategy

The executor uses a maker-first approach with time-aware escalation.

Execution Modes

Mode	Trigger	Behavior
WAIT	Position blocker active	No trade this window
MAKER_PATIENT	> 60s to close	Limit order at fair value - 1¢, 15s timeout
MAKER_AGGRESSIVE	30-60s to close	Limit order at fair value - 1¢, 10s timeout

Mode	Trigger	Behavior
TAKER_NOW	< 30s to close	Skip maker, submit at best ask
PANIC_CAPTURE	High urgency, endgame	Submit at 99¢ (maximum price)

Maker-to-Taker Escalation

1. Place maker order at `fair_value - 1¢`
2. Poll every 2 seconds for fill
3. If timeout reached without fill:
 - Cancel maker order
 - Re-fetch orderbook for current best ask
 - Validate price still in [80¢, 99¢]
 - Submit taker order at best ask

UUID Persistence

Each order gets a `client_order_id` (UUID4) written to SQLite before API submission. This ensures crash recovery — if the bot restarts mid-order, it can reconcile using the persisted UUID.

3.6 Position Sizing

Quarter-Kelly Formula

The Kelly criterion maximizes long-run growth rate. The bot uses a quarter-Kelly fraction for safety:

$$f = 0.25 \times \frac{b \times p - q}{b}$$

where:

- $b = \frac{100 - \text{price}}{\text{price}}$ (net odds)
- p = win probability
- $q = 1 - p$

The dollar position is:

$$\text{contracts} = \left\lfloor f \times \frac{\text{bankroll}}{\text{price}} \right\rfloor$$

clipped to [1, 5] contracts.

Drawdown Scaling

Balance vs. Starting	Sizing Adjustment
90%	Full quarter-Kelly
80–90%	Half of quarter-Kelly

Balance vs. Starting	Sizing Adjustment
< 80%	Quarter of quarter-Kelly

This creates a geometric de-risking curve that preserves capital during losing streaks.

Part 4: Risk Management

Position Sizing Controls

- **Quarter-Kelly:** Conservative fraction ($0.25\times$) of the theoretically optimal bet size
- **Drawdown scaling:** Size halved below 90% of starting balance, quartered below 80%
- **Hard limits:** Maximum 5 contracts per trade; maximum risk per trade capped as percentage of bankroll

Market Selection Controls

- **Single-asset-per-window:** Only one asset traded per 15-minute window (the one with highest edge), preventing correlated exposure
- **Price range guardrails:** Only trade contracts priced 80–99¢ — below 80¢ implies too much uncertainty; above 99¢ offers insufficient reward
- **Minimum edge threshold:** Fee-adjusted edge must exceed 0.25% after taker fees

Model Sanity Controls

- **Z-score limit:** Refuse markets where $|z| > 8$ (extreme inputs suggest data issues)
- **Model-market discrepancy:** If the model estimates >90% probability but the market prices below 75¢, refuse (the model may be missing material information)
- **Dynamic probability cap:** Time-dependent ceiling (90–93%) prevents overconfidence regardless of model output

Execution Controls

- **Maker-first:** Reduces fees by 75% compared to taker orders when fills are obtained
 - **Price re-validation:** After maker timeout, the system re-fetches the orderbook and re-validates the price range before submitting a taker order
 - **UUID persistence:** Order IDs written to disk before API submission, enabling crash recovery without duplicate orders
-

Part 5: Performance & Observations

This section contains live statistics from the bot's observation database, updated on each whitepaper build.

Overview

Metric	Value
Total markets evaluated	0
Total markets settled	0
Observation period	N/A
Assets tracked	BTC, ETH, SOL, XRP

Filter Pipeline Breakdown

Of all evaluated markets, here is how they were classified:

Filter Stage	Count	Percentage
Low probability	0	0%
No orderbook	0	0%
No best ask	0	0%
Price out of range	0	0%
Insufficient edge	0	0%
Zero sizing	0	0%
Strategy wait	0	0%
Candidate (passed all filters)	0	0%

Hypothetical Performance

Based on observation-mode tracking of markets that passed all filters:

Metric	Value
Total hypothetical trades	0
Observation P&L (cents)	0
Win rate	N/A

Win Rate by Price Bucket

Entry Price	Trades	Wins	Win Rate
80–84¢	0	0	—
85–89¢	0	0	—
90–94¢	0	0	—
95–99¢	0	0	—

Part 6: Infrastructure

Deployment

- **Host:** DigitalOcean droplet (Ubuntu 24.04)

- **Runtime:** Python 3, virtualenv
- **Process manager:** systemd (kalshi-bot service)
- **Startup:** `start.sh` activates venv, sources `.env`, launches `bot.py`
- **Auto-deploy:** Pushing to `main` triggers a GitHub Action that SSHes into the VPS, pulls the latest code, syntax-checks `bot.py`, and restarts the service

Data Persistence

SQLite (state.db)

The primary state store uses SQLite in WAL (Write-Ahead Logging) mode for crash resilience:

Table	Purpose
<code>positions</code>	Active positions (ticker, asset, side, count, avg price)
<code>pending_orders</code>	Orders awaiting fill (with UUID <code>client_order_id</code>)
<code>settled_trades</code>	Completed trades with P&L
<code>rejected_opportunities</code>	Markets rejected with reason and model state
<code>evaluated_opportunities</code>	Every market evaluation with filter stage (UNIQUE on ticker + stage)

JSONL Journals

Eight append-only journal files provide a complete audit trail:

1. **scan_journal** — Every tick: price snapshots, orderbook depth
2. **opportunity_journal** — Every market evaluation with filter stage
3. **rejection_journal** — Settlement outcomes for rejected opportunities
4. **trade_journal** — Order fills with entry/exit details
5. **settlement_journal** — Outcome determination and P&L
6. **order_journal** — Full order lifecycle (create, cancel, fill)
7. **execution_journal** — Maker-to-taker escalation events
8. **performance_journal** — Session summaries and strategy counts

Firebase Real-Time Dashboard

A Firebase integration provides a live web dashboard showing:

- Current positions and P&L
- Active market evaluations
- Volatility regime indicators
- Order flow signals

Observation Mode

The bot supports a full observation mode (`OBSERVATION_MODE = True`) where it runs the complete evaluation pipeline — volatility estimation, probability calculation, edge detection, and position sizing — but does not submit any orders. All hypothetical trades are logged to the database and journals, enabling strategy validation before committing capital.

Generated: N/A
